CMSC 35200 Lab 3: Smaller, Faster Neural Networks with Pi

Evidence report (Parts A–C)

Tushar Sonthalia (sonthalt)

November 2026

I compressed three laptop-scale reference models: an MLP and a CNN on Fashion-MNIST, and a two-layer GCN on Cora. I used four sparsity methods (S1–S4) and five reduced-precision formats, then ran a 12-condition sparse×INT8 factorial. Everything was measured on one CPU: accuracy, on-disk bytes, memory, and warmed latency. This report collects the evidence. The one-page interpretation is in `memo.pdf`.

Every number in a table comes from `results/runs.jsonl`. The tables are generated by `scripts/make_report_tables.py`. Run IDs are cited by their unique 6-hex suffix, which is unique across all 386 records. Superseded or excluded records are listed, with reasons, in `results/ANNOTATIONS.md`, and are never used.

1 Setup and reproducibility

- **Hardware/software.**
  - Apple M4 Pro, 12 cores, 24 GB, macOS 27.0.
  - **CPU only** for every arm; `torch.set_num_threads(4)`, interop threads 1.
  - Python 3.12.13, PyTorch 2.15.0, torchvision 0.29.2, torch_geometric 2.8.1, NumPy 2.5.4, locked with `uv`.
  - Quantized engine: `qnnpack`, the only one available on this machine.
- **Data.**
  - Fashion-MNIST: 55k train / 5k stratified validation / 10k official test, split seed 12345, normalization from the training split only.
  - Cora: public Planetoid split (140 / 500 / 1000 nodes); the graph is never modified.
  - Splits are saved with SHA-256 hashes in `splits/` and verified by every command.
- **Seeds.** Seed 0 produces the canonical dense parent and the full pilot. Seeds 1–3 are used for replicas and confirmations.
- **Latency protocol.**
  - `inference_mode`, model in `eval()`, 20 warm-up and 200 timed iterations.
  - Reported: median and IQR at batch 1 and batch 256 (Fashion-MNIST), or one full-graph forward (Cora).
  - Peak RSS is measured in a fresh subprocess.
  - Bytes are `os.path.getsize` of the saved file.
- **Compute.** Sum of record wall times: about 60 min (Part A 11.7, Part B 44.6, Part C 3.8 min).
- **Tests.** 153 tests pass:
  - an overfit test and a parameter-count test per architecture;
  - eval/bench never update weights, and split integrity;
  - masks stay zero after fine-tuning, and S2 shapes are consistent;
  - reload reproduces logits, and compact logits equal dense-with-zeros logits;
  - N:M block structure, INT4 matches a reference from disk, and INT8 accounting.
- **Replay.** One command per part, listed in `README.md`.

2 Frozen design and its enforcement

`predictions.md` froze the design before the governing runs:

- architectures, budgets, the early-stopping rule, and seeds;
- the four method definitions, levels, and N:M patterns;
- the fine-tuning budget and the calibration rule;
- two validation-only selection rules: confirmatory points (B.3) and Part C operating points (C.1).

The file is append-only.

`scripts/predictions_provenance.py` compares each section, line by line, with what the coding agent read in the unedited transcripts. **All 15 sections appear verbatim before the first run they govern**:

- Part A: 14:10 vs first training run 14:18;
- B.1–B.5: 14:53 vs first pruning run 16:05;
- C.1–C.3: 20:39 vs first Part C run 20:50 (all UTC, 9 Nov).

The table is in `results/predictions_provenance.md`.

Two later sections are dated amendments that state their trigger:

- **B.6** adds a *dense fine-tuned control* (`dense_ft`: same parent, same budget, no pruning). It was added after the smoke runs showed pruned models beating the parent, and before the pilot matrix.
- **B.7** records the mechanical application of the selection rule. It compares eligibility in integer counts of correct validation examples. Comparing float32 accuracies would have wrongly rejected GCN S2 at 50%: 390/500 is exactly the threshold.

3 Part A: dense reference models

Table 1: Dense baselines. Accuracy is for the seed-0 parent; seeds 1–3 are replicas (test mean ± sd). Epochs are best/stopped. Bytes are the on-disk `state_dict`. MACs are computed from layer shapes (per image; per full graph for the GCN, dense-X convention). Latency is the median (IQR) in ms. ΔRSS is the model's memory over the process baseline.

Arch	Val	Test	Test s1–3	Ep.	Params	Bytes	MACs	bs1 / bs256 or graph	ΔRSS MB	Runs
MLP	0.8986	0.8929	0.8909±0.0032	16/21	235,146	943,589	234,752	0.013 (0.001) / 0.132 (0.005)	2.5	69a724, 93cf88
CNN	0.9182	0.9087	0.9144±0.0009	20/25	117,066	471,797	1,697,280	0.093 (0.005) / 7.079 (0.114)	56.5	1a54cb, adedb7
GCN	0.8000	0.8080	0.8163±0.0081	93/143	23,063	94,741	62,697,392	– / 1.053 (0.040)	60.6	bdf9a8, 180141

All three baselines met their sanity targets (MLP ≥82%, CNN ≥87%, GCN ≥65%) on the first pass, with the frozen hyperparameters.

Seed-0 training took 0.7 s (GCN, `bdf9a8`), 6.6 s (MLP, `69a724`) and 156 s (CNN, `1a54cb`). That is the predicted order, CNN > MLP > GCN. The CNN does about 7× the MLP's MACs per image.

The Part A bench rows were invoked through the test suite (see `ANNOTATIONS.md`). The same parents were re-benchmarked in Part C under the same protocol: MLP 0.130 ms (`844c70`), CNN 6.73 ms (`185cdf`), GCN 1.07 ms (`bcec0f`).

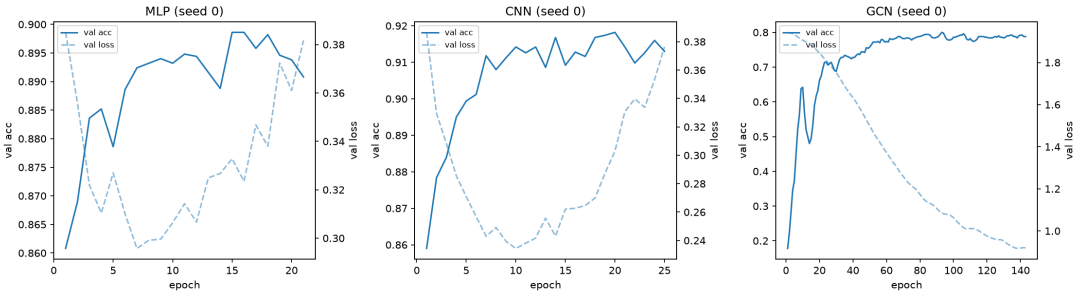


Figure 1: Learning curves of the dense baselines; the selected checkpoint is the best validation epoch.

4 Part B: four forms of sparsity

Methods as implemented. The eligible tensors are every Linear, Conv2d and GCNConv weight, including the classifier. Biases are never pruned.

- **S1:** global $|w|$ magnitude pruning.
- **S2:** layer-wise L1 group pruning:
  - groups are MLP hidden units, CNN conv channels plus FC1 units, and GCN hidden features;
  - each layer keeps $\max(1, \text{round}((1-s)n))$ groups;
  - the model is *physically rebuilt* smaller.
- **S3:** global *unstructured* $|w \odot \partial L/\partial w|$:
  - the gradient comes from a fixed calibration set: 1,024 stratified *training* images, or the 140 Cora training nodes;
  - no validation or test labels are used.
- **S4:** $N:M$ along each row's reduction dimension:
  - patterns 3:4, 2:4, 1:4 and 1:10 for 25/50/75/90%;
  - a partial final block keeps $\lceil Nr/M \rceil$ weights.

Fine-tuning and paths.

- Every arm gets the same fine-tuning budget: 3 epochs at lr 5×10^{-4}, or 50 GCN epochs.
- Masks are re-applied after every step.
- Each arm is measured after pruning and after fine-tuning, on two runtime paths:
  - dense modules holding zeros;
  - a compact path: the rebuilt model for S2, `torch.sparse_csr` with int32 indices and `sparse.mm` for S1/S3, and a packed $N:M$ file for S4.

Pilot. The full $3\times4\times4$ matrix ran on seed 0, plus three dense_ft controls. All **51/51 arms completed** with no failures and no reruns, in 755 s. No cell is "not applicable".

Table 2: Pilot matrix (seed 0). For each target: achieved eligible-weight sparsity, validation accuracy right after pruning, and after fine-tuning. S2's achieved sparsity exceeds its group target because removing a group also removes downstream weights. The dense_ft controls reached val MLP 0.906 (d5b93e), CNN 0.918 (2ee387), GCN 0.798 (46d95b). The dense parents are 0.899 / 0.918 / 0.800.

		25%			50%			75%			90%		
Arch	Method	ach	pre	post	ach	pre	post	ach	pre	post	ach	pre	post
MLP	S1 magnitude	0.25	0.899	0.906	0.50	0.890	0.904	0.75	0.807	0.903	0.90	0.604	0.892
MLP	S2 structured	0.28	0.887	0.902	0.53	0.761	0.897	0.78	0.504	0.883	0.91	0.157	0.846
MLP	S3 saliency	0.25	0.895	0.903	0.50	0.885	0.900	0.75	0.838	0.900	0.90	0.670	0.889
MLP	S4 N:M	0.25	0.894	0.903	0.50	0.858	0.900	0.75	0.739	0.898	0.90	0.329	0.874
CNN	S1 magnitude	0.25	0.916	0.921	0.50	0.908	0.919	0.75	0.852	0.919	0.90	0.538	0.904
CNN	S2 structured	0.43	0.798	0.914	0.75	0.719	0.904	0.93	0.307	0.877	0.99	0.097	0.805
CNN	S3 saliency	0.25	0.918	0.921	0.50	0.897	0.919	0.75	0.797	0.916	0.90	0.367	0.899
CNN	S4 N:M	0.25	0.867	0.918	0.50	0.716	0.920	0.75	0.276	0.893	0.90	0.171	0.848
GCN	S1 magnitude	0.25	0.800	0.796	0.50	0.800	0.798	0.75	0.788	0.794	0.90	0.780	0.794
GCN	S2 structured	0.25	0.746	0.794	0.50	0.534	0.780	0.75	0.268	0.434	0.88	0.114	0.152
GCN	S3 saliency	0.25	0.800	0.796	0.50	0.788	0.800	0.75	0.782	0.792	0.90	0.764	0.796
GCN	S4 N:M	0.25	0.720	0.794	0.50	0.476	0.776	0.75	0.146	0.484	0.90	0.118	0.292

Confirmation. The frozen B.3 rule reads only validation accuracy:

1. Keep the levels whose post-fine-tune accuracy is within τ of the dense parent, counted in correct validation examples. τ is 1 pp for MLP/CNN and 2 pp for the GCN.
2. Pick the highest sparsity among them.

Each selected point was rerun with seeds 1–3 from the same parent: 45/45 arms ok in 645 s. No confirmed mean fell below its threshold. The largest seed sd was 0.53 pp (GCN S4, test).

Table 3: Confirmed operating points (seeds 1–3, mean ± sd). Δ columns compare mean validation accuracy with the dense parent and with the dense_ft control (pp). KB and ms are given as *dense-with-zeros / compact*. Latency is the median at batch 256, or the full graph for the GCN.

Arch	Method	Target	Ach.	Val	Test	Δpar	Δft	KB dz / cpt	ms dz / cpt (path)	Runs
MLP	S1 magnitude	0.90	0.900	0.8935±0.0018	0.8859±0.0007	-0.51	-0.95	943.6 / 193.3	0.135 / 1.949 (CSR)	983f78, 05c061, b977d8
MLP	S2 structured	0.50	0.535	0.8985±0.0012	0.8928±0.0010	-0.01	-0.45	943.6 / 440.7	0.167 / 0.067 (rebuilt)	43c7f6, cd6e77, 9e86f2
MLP	S3 saliency	0.90	0.900	0.8889±0.0021	0.8798±0.0014	-0.97	-1.40	943.6 / 193.4	0.148 / 1.846 (CSR)	117470, 1cc2c2, b70e70
MLP	S4 N:M	1:4	0.750	0.8947±0.0009	0.8870±0.0020	-0.39	-0.82	943.6 / 296.7	0.148 / 0.146 (dense fallback)	8622c7, 7f0255, 1a45fb
CNN	S1 magnitude	0.75	0.750	0.9203±0.0009	0.9144±0.0005	+0.21	-0.10	471.8 / 239.2	6.797 / 30.52 (CSR)	4cf175, 6c4afe, d269ed
CNN	S2 structured	0.25	0.435	0.9139±0.0006	0.9104±0.0010	-0.43	-0.74	471.8 / 268.5	6.787 / 5.241 (rebuilt)	ae7582, d7e1bd, 12e9c1
CNN	S3 saliency	0.75	0.750	0.9157±0.0024	0.9093±0.0010	-0.25	-0.57	471.8 / 239.4	6.732 / 39.29 (CSR)	ecb565, c34a0c, 735f6a
CNN	S4 N:M	2:4	0.500	0.9184±0.0016	0.9116±0.0019	+0.02	-0.29	471.8 / 296.1	6.718 / 6.768 (dense fallback)	6f0393, 7a1e2f, f75925
GCN	S1 magnitude	0.90	0.900	0.7913±0.0012	0.8007±0.0051	-0.87	-0.47	94.7 / 21.8	1.053 / 4.363 (CSR)	d2efb5, 9bdca2, 8e3dd0
GCN	S2 structured	0.50	0.500	0.7807±0.0012	0.7720±0.0030	-1.93	-1.53	94.7 / 48.6	1.056 / 0.895 (rebuilt)	e33eed, b9f801, 513069
GCN	S3 saliency	0.90	0.900	0.7947±0.0012	0.8023±0.0021	-0.53	-0.13	94.7 / 21.8	1.061 / 4.352 (CSR)	ad1dc9, b5e43b, fb277b
GCN	S4 N:M	3:4	0.250	0.7940±0.0020	0.8040±0.0053	-0.60	-0.20	94.7 / 89.2	1.063 / 1.064 (dense fallback)	a060b6, 2a8900, 8116cd

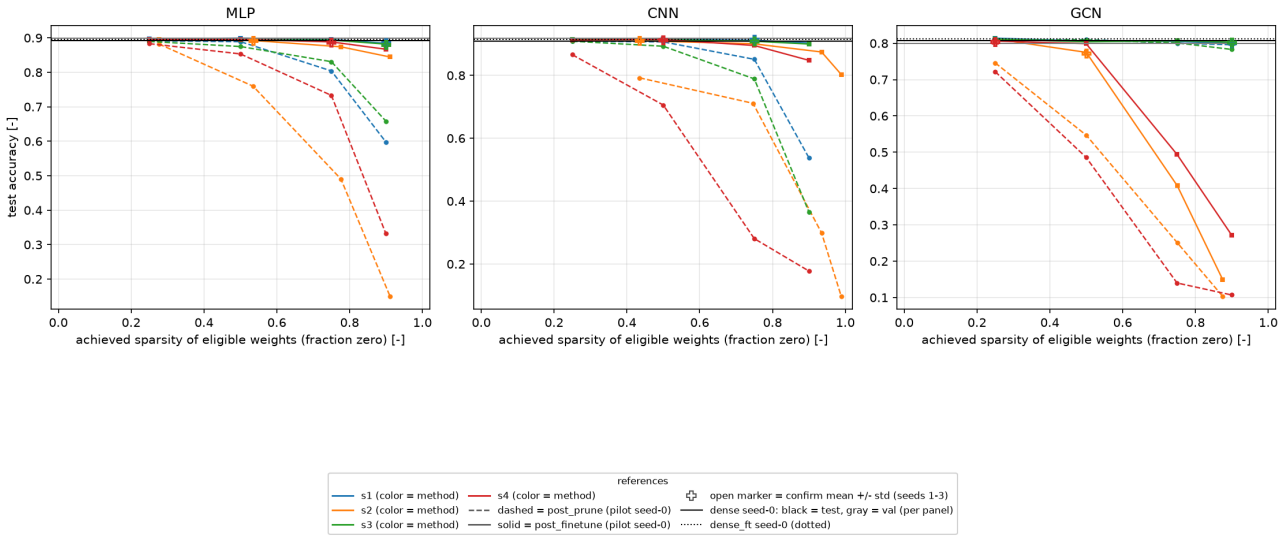


Figure 2: Test accuracy vs achieved sparsity (pilot seed 0): dashed lines are after pruning, solid lines after fine-tuning. Open markers are confirmation means ± sd.

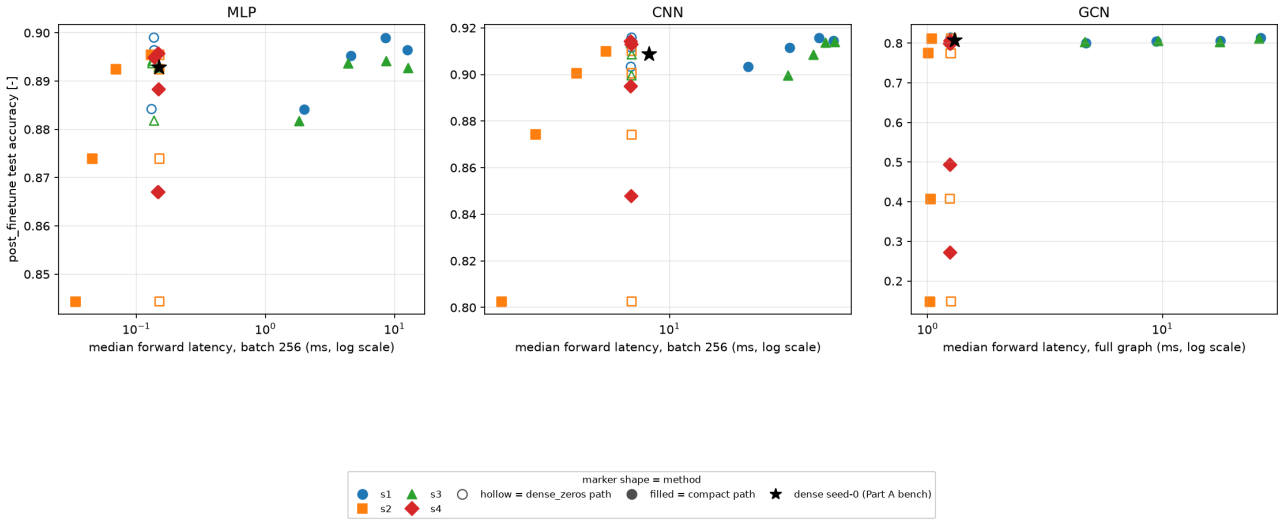


Figure 3: Latency vs accuracy (pilot, after fine-tuning). Hollow markers are dense-with-zeros, filled markers are compact. Only S2's compact (rebuilt) points move left of the dense baseline.

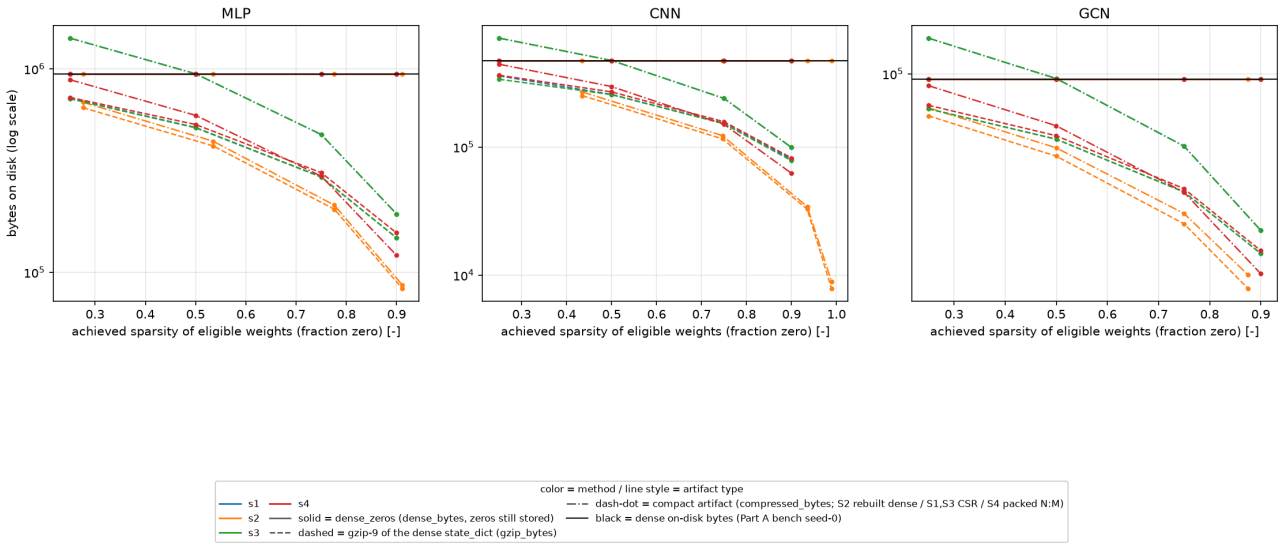


Figure 4: On-disk bytes vs sparsity for the dense-with-zeros file, its gzip, and the compact artifact.

What the measurements show.

- **S2 is the only method that is faster.** The rebuilt models run at 0.067 vs 0.167 ms (MLP), 5.24 vs 6.79 ms (CNN) and 0.89 vs 1.06 ms (GCN), using the confirmation medians.
- **Unstructured CSR is slower everywhere.**
  - At the confirmed points it is 4–14× slower than dense. Across the pilot it ranges from 3.2× (CNN S1 90%, `d5fa24`) to 91× (MLP S1 25%, `704ff3`).
  - The CSR file beats the dense file only at ≥50% sparsity: 944,648 vs 943,589 B at 50% (`f2b98a`).
- **S4 has no semi-structured kernel on this CPU.**
  - `to_sparse_semi_structured` rejects CPU tensors, so the dense fallback runs at dense speed.
  - The packed file still shrinks: MLP 2:4 is 590,165 B vs 943,589 B (`ddb71a`).
- **The GCN depends on structure.**
  - Unstructured S1/S3 hold about 0.79 validation accuracy even at 90%.
  - S2 and S4 collapse above 50%: S2 at 75% reaches 0.434 (`94be4d`) and 1:4 reaches 0.484 (`064215`). Both remove width from a 16-unit bottleneck.
- **Extra epochs, not pruning, explain the gains over the parent.** The dense_ft control shows that post-fine-tune accuracy above the parent comes from the additional training. Against dense_ft, every confirmed point is at or below the control.

Per-layer sparsity for every arm is in `results/per_layer_sparsity.md`. Failure accounting is in `results/failure_accounting.md`: all 9 Part B failures are smoke and debug rows from before the pilot.

5 Part C: reduced precision

Backend support (`results/precision_support.md`):

- `qnnpack` is the only engine.
- `quantize_dynamic` and FX `prepare_fx`/`convert_fx` produce native quantized modules.
- FP16 and BF16 run natively, with no upcast.
- PT2E and torchao are unavailable.
- PyG's GCN aggregation cannot be quantized, so only the two projections run in INT8 and aggregation stays FP32.
- Simulated INT4 is per-output-channel symmetric $[-8, 7]$, packed two values per byte. It is dequantized once

(cached) and computed in FP32, so its dequantization cost is *outside* the timed region (`ANNOTATIONS.md`). **Calibration** uses the same 1,024 stratified training images (inputs only), or the Cora full-graph features. The best INT8 method per architecture was chosen by the frozen rule: higher validation accuracy, with ties within one example going to lower latency.

Table 4: Standalone precision results from each seed-0 parent. Precision is given as stored/compute/accumulator. KB is the real artifact file. ms are medians at batch 1 and at batch 256 (or the full graph). Frac is the fraction of MACs that executed in reduced precision.

Arch	Format	Precision	Val	Test	KB	bs1	bs256/graph	Frac	Run
MLP	FP32 (parent)	fp32/fp32/fp32	0.8986	0.8929	944.3	0.012	0.130	0.00	844c70
	FP16	fp16/fp16/fp16	0.8982	0.8929	473.9	0.052	0.624	1.00	981f86
	BF16	bf16/bf16/bf16	0.8988	0.8927	473.9	0.054	0.851	1.00	ba7522
	INT8 dynamic	int8/int8/int32	0.8984	0.8927	242.5	0.096	0.519	1.00	7df957
	INT8 static (FX)	int8/int8/int32	0.8984	0.8931	243.0	0.099	0.404	1.00	25d6f7
	INT4 simulated	int4/fp32/fp32	0.8914	0.8873	127.1	0.016	0.143	0.00	143754
CNN	FP32 (parent)	fp32/fp32/fp32	0.9182	0.9087	472.6	0.093	6.725	0.00	185cdf
	FP16	fp16/fp16/fp16	0.9182	0.9087	238.3	0.328	18.25	1.00	34979c
	BF16	bf16/bf16/bf16	0.9180	0.9087	238.3	0.342	19.65	1.00	12ace5
	INT8 dynamic	int8/int8/int32	0.9182	0.9082	163.2	0.169	7.292	0.06	9e1d6e
	INT8 static (FX)	int8/int8/int32	0.9154	0.9082	127.3	0.182	3.354	1.00	f082d1
	INT4 simulated	int4/fp32/fp32	0.9006	0.8969	69.8	0.097	6.928	0.00	002cb7
GCN	FP32 (parent)	fp32/fp32/fp32	0.8000	0.8080	95.4	–	1.070	0.00	bcec0f
	FP16	fp16/fp16/fp16	0.8000	0.8080	49.1	–	1.684	1.00	4f8355
	BF16	bf16/bf16/bf16	0.8000	0.8080	49.2	–	1.819	1.00	458226
	INT8 dynamic	int8/int8/int32	0.7960	0.8080	28.9	–	1.535	1.00	d73a16
	INT8 static (FX)	int8/int8/int32	0.7940	0.8090	30.0	–	1.274	1.00	312d78
	INT4 simulated	int4/fp32/fp32	0.7860	0.8040	17.1	–	1.067	0.00	2f7015

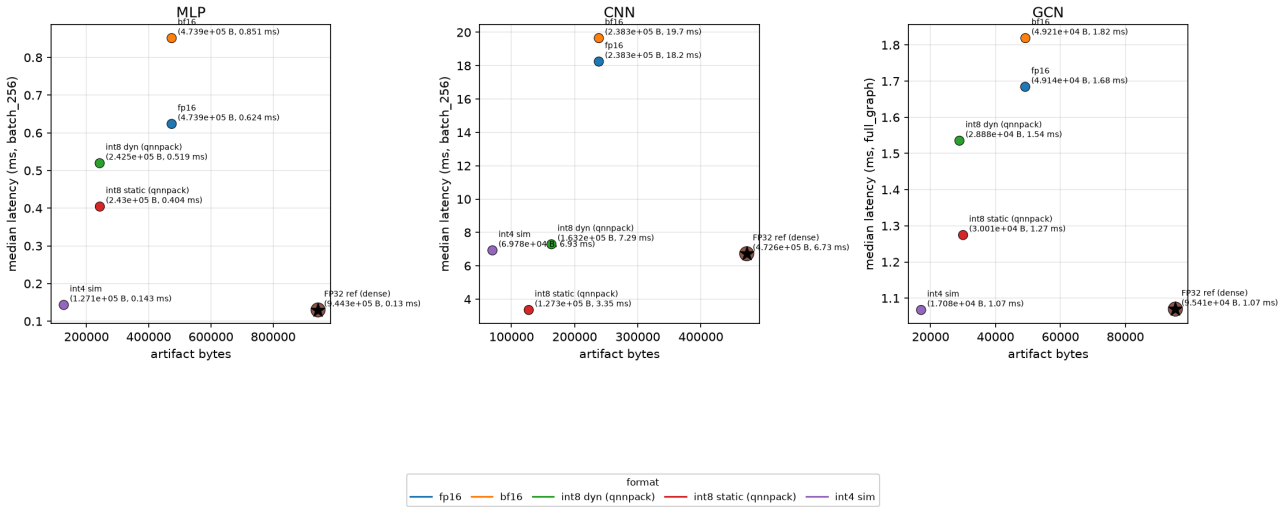


Figure 5: Artifact bytes vs latency per format; the star is the FP32 parent.

Factorial. The sparse operating points were chosen by the frozen C.1 rule: among the confirmed points that pass the validation check, the smallest compact artifact. They are: MLP: S1 magnitude at 0.90 sparsity with

int8 static; CNN: S1 magnitude at 0.75 sparsity with int8 dynamic; GCN: S3 saliency at 0.90 sparsity with int8 static.

Table 5: Sparse×INT8 factorial. For INT8 rows, NNZ is nonzeros before / after quantization. Latency is the median at batch 256, or the full graph for the GCN.

Arch	Condition	Val	Test	KB	NNZ	ms	Frac	Run
MLP	dense FP32	0.8986	0.8929	944.3	235,146	0.130	0.00	844c70
	sparse FP32	0.8924	0.8842	944.3	23,869	0.133	0.00	ae0892
	dense INT8	0.8984	0.8931	243.0	235,146 / 227,255	0.406	1.00	e2d198
	sparse+INT8	0.8936	0.8854	243.0	23,869 / 23,469	0.402	1.00	5ad684
CNN	dense FP32	0.9182	0.9087	472.6	117,066	6.725	0.00	185cdf
	sparse FP32	0.9194	0.9115	472.6	29,406	6.757	0.00	08af14
	dense INT8	0.9182	0.9082	163.2	117,066 / 100,646	7.357	0.06	3976fe
	sparse+INT8	0.9192	0.9117	163.2	29,406 / 23,832	7.137	0.06	5e3be7
GCN	dense FP32	0.8000	0.8080	95.4	23,063	1.070	0.00	bcec0f
	sparse FP32	0.7960	0.8030	95.4	2,327	1.081	0.00	0ef243
	dense INT8	0.7940	0.8090	30.0	23,063 / 18,972	1.237	1.00	5b95aa
	sparse+INT8	0.7960	0.8040	30.0	2,327 / 2,304	1.210	1.00	24dba8

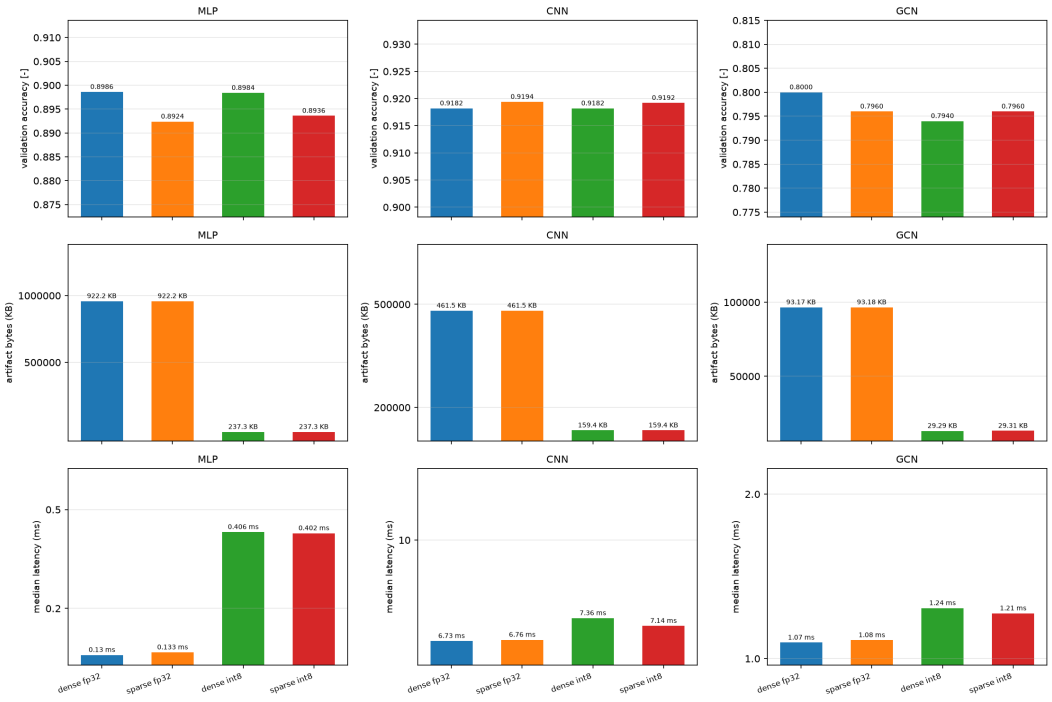


Figure 6: Factorial: validation accuracy, artifact size, and latency for the four conditions per architecture.

What the measurements show.

- **Accuracy.** FP16/BF16 lose at most 0.04 pp. INT8 loses at most 0.6 pp (GCN static). Simulated INT4 loses 0.72 / 1.76 / 1.4 pp for MLP / CNN / GCN.
- **Size.** INT4 is smallest, at 13–18% of FP32. INT8 is 26–35% (CNN dynamic is the largest, at 34.5%) and FP16/BF16 about 50%.

- **Latency.** Only one reduced format is faster than FP32: CNN static INT8, at 3.35 vs 6.73 ms (2.0×) at batch 256. It is the only format whose convolutions execute in INT8.
  - CNN dynamic INT8 quantizes only 6.1% of MACs and is slower (7.29 ms).
  - At batch 1 even static INT8 is slower: 0.182 vs 0.093 ms.
  - FP16/BF16 are 1.6–6.5× slower on every model (GCN 1.6–1.7×, CNN 2.7–2.9×, MLP 4.8–6.5×).
- **Composition.** Sparsity and INT8 compose on accuracy, but not on size or speed.
  - Zeros survive symmetric quantization; MLP sparse nonzeros go from 23,869 to 23,469.
  - The INT8 files store them densely, so sparse+INT8 is the same size as dense INT8: 243,001 vs 242,981 B.
  - No sparse INT8 kernel exists, so latency is the same as well.
- **Reload.** Every artifact is reloaded from disk and reproduces its logits (max |diff| = 0).

6 Predictions vs outcomes

Table 6: Selected frozen predictions from `predictions.md` against the measured outcome.

Prediction	Outcome	Verdict
A.2 Params and bytes: MLP > CNN > GCN	235k/117k/23k params; 944/472/95 KB	held
A.2 Training time: CNN > MLP > GCN	156 / 6.6 / 0.7 s (`1a54cb`, `69a724`, `bdf9a8`)	held
A.2/B.5 GCN most sensitive to sparsity	Least sensitive unstructured (S3 90% within 0.13 pp of dense_ft); most sensitive structured (S2 75% = 0.434)	split
B.5 S1 GCN at 90%: 3–5 pp below dense	0.791 / 0.795 val (S1/S3), 0.5–0.9 pp below the parent (Table 3)	missed
B.5 Zeros do not change dense-path size/latency	Dense-with-zeros file and latency unchanged in every arm	held
B.5 CSR smaller only above 60–67%	Break-even at 50% with int32 indices (`f2b98a`)	missed
B.5 Sparse matmul slower at every level	3.2–91× slower (`d5fa24`, `704ff3`)	held
B.5 Only S2 improves size and latency	S2 is the only faster compact path	held
B.5 S4: no CPU kernel; pack smaller at ≥2:4	Dense fallback; 2:4 pack is 63% of dense (`ddb71a`)	held
C.2 INT4 minimizes file size	13–18% of FP32	held
C.2 No reduced format beats FP32 at batch 1	True for all formats and models	held
C.2 At most a modest INT8 gain at batch 256	CNN static INT8 2.0× faster (`f082d1`)	stronger
C.2 INT4 slowest	About equal to FP32, since dequantization is cached outside timing	missed
C.2 INT4 loss: MLP 0.5–1, CNN 1–2, GCN 1–3 pp	0.72 / 1.76 / 1.4 pp	held
C.2 Sparse+INT8 does not compose on latency	Same latency as dense INT8	held

7 Agent process, corrections, and provenance

The agent. Pi (Qwen3.8-27B, course endpoint) implemented the code and tests, and ran every experiment. It ran in one non-interactive session per task:

- A1–A2, B1, B1b, B2, B3, B3b;
- C1, C1b, C2–C5.

Every prompt is in `transcripts/prompts/`, and every complete transcript is in `transcripts/`.

Corrections after review. Every correction is recorded in the follow-up prompts and in `ANNOTATIONS.md`. The superseded rows stay in the log.

- **Part A:** the CNN and GCN MAC formulas; file bytes; RSS that measured the dataset; a no-update test that bypassed the real code.
- **Part B:**

  - CSR multiplied in the slow dense×sparse orientation, with a Python loop over the batch;
  - int64 CSR indices;
  - a 3-index $N:M$ pack;
  - S2's two paths being the same model;
  - tests writing into `results/`;
  - empty figures.
- **Part C:**
  - INT8 emulated with Python int64 matmuls, replaced by native `qnnpack`;
  - INT4 scoring chance, because it was built from random weights;
  - a 1.8 KB "INT8" file with no weights;
  - INT8 nonzero counts of 0, and GCN INT8 accounting that saw only biases;
  - nested-pickle INT8 files;
  - a Python 3.13 environment (all Part C results were rerun on 3.12.13).

The endpoint truncated several replies. Those sessions were resumed with logged "continue" prompts.

Accounting. 386 records in total: Part A 56 ok / 9 failed, Part B 198 ok / 9 failed, Part C 97 ok / 17 failed. Failed and superseded records stay in the log, each with a classified reason in `ANNOTATIONS.md`. One recorded field, `sparse_int8_storage_bytes`, is invalid and excluded: it scales with all weights, not with nonzeros.